

Database and Platform Documentation

A system-level guide to CheMatSustain's scientific data model, production architecture, governed access paths, and one complete test implementation from Excel parser to interactive viewer.

CheMatSustain Research Database

Document version
1.0

Prepared
14 September 2026

Repository baseline
f831ba3

Production portal
database.eurskem.com

Contents and Scope

Purpose. This document explains the entire CheMatSustain database in its application context. “Database” therefore includes the PostgreSQL schema, normalized scientific JSON, persistent source files, APIs, viewers, access controls, audit records, and operational lifecycle that make the data usable and governable.

1. Overview

1. Mission and capabilities
2. Scientific test inventory
3. Data organization

2. Architecture and technology stack

1. Architecture image
2. Component responsibilities
3. Broader research context
4. Deployment and security

3. Database design

1. Storage model
2. Table catalogue
3. Relationships
4. Release and tenancy controls
5. Lifecycle and retention

4. User process flows

1. Public viewer
2. Team user
3. Administrator
4. API client

5. Worked implementation: TB

1. Workbook contract
2. Backend parser
3. Upload and persistence
4. Read projection
5. Frontend viewer
6. Extension checklist

6. Operations and maintenance

7. Glossary and source map

Audience

Scientific and project users

Understand how uploaded laboratory workbooks become searchable reports and which publication choices affect public visibility.

Developers and maintainers

Locate the relevant model, parser, service, route, viewer, migration, test, deployment, and security logic.

Administrators and data owners

Understand account classes, resource assignments, publication flags, API credentials, access history, and safe data lifecycle.

Auditors and reviewers

Trace controls from browser/API identity to authorization, PostgreSQL policy, field projection, audit event, retention, and CI/CD gate.

Baseline and limitations. This is an implementation document generated from repository state `f831ba3`. It describes code and migration intent; it is not a live production data dictionary with row counts or confidential values. Some migration-era columns retain `keycloak_*` / `oauth_*` names, but active authentication is local session authentication and locally issued API client credentials.

1. Platform Overview

CheMatSustain is a research data portal for selected chemicals and nanomaterials. It converts heterogeneous test-specific Excel workbooks into a consistent five-section data contract, stores the result with ownership and publication metadata, and presents it through both human-oriented scientific viewers and scope-controlled machine APIs.

19

registered scientific test types

5

independently releasable data sections

3

interactive roles: public, team, admin

1.1 Core capabilities

- ✓ Upload and retain source Excel workbooks for scientific traceability.
- ✓ Parse each assay/instrument format with a dedicated Python parser.
- ✓ Normalize parser output into a stable JSON contract.
- ✓ Browse data by work package, CMS material identifier, test category, and test type.
- ✓ Render specialized tables, charts, statistics, images, and CSV downloads.
- ✓ Publish a test while independently selecting which scientific sections are public.
- ✓ Support authenticated public viewers, internal team users, administrators, and machine clients.
- ✓ Apply resource grants, API scopes, tenant context, access logs, and operational security gates.

1.2 Canonical scientific record

Section	Purpose	Typical content	Public control
test_details	Experimental context	Work package, material, preparation, instrumentation, cell line, treatment	release_test_details
raw_data	Observed/source measurements	Runs, spectra, time series, plates, replicate observations	release_raw_data
processed_data	Derived measurements	Normalized values, fitted curves, calculated metrics	release_processed_data
final_results	Reported outcome	Summary metrics, means, standard deviations, acceptance results	release_final_results
statistical_analysis	Statistical interpretation	ANOVA, Brown–Forsythe, Tukey comparisons, significance	release_statistical_analysis

Public header exception. Public reports always retain a narrow identification projection from `test_details` : full test name, acronym, type, endpoint, endpoint outcome, SOP, and ERM identifier. This does not release the Test Conditions tab or any other test-detail fields.

1.3 Registered tests

MTT	DLS	FTIR	HR-STEM	UV-VIS	ZETA	SIMS	ROS	TB	UPS	XPS	XRD	DSC	TGA	MNT	TB-Microfluidic
Rotifier	WaterFlea	Algae													

The spelling `TB-Microfluidic` and `Rotifier` are canonical identifiers retained for database/API compatibility.

2. Architecture and Technology Stack

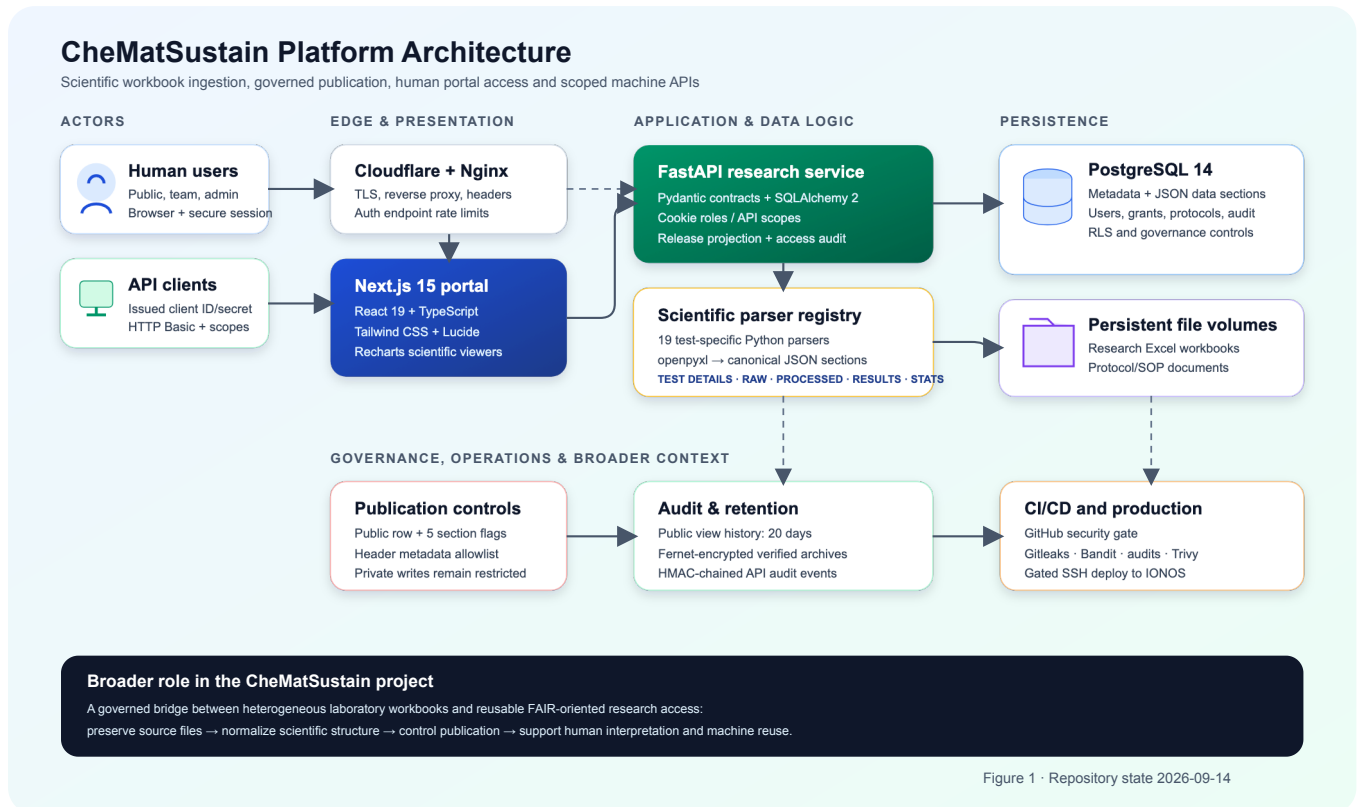


Figure 1. Production-oriented component architecture. Solid arrows represent principal request/data paths; dashed arrows represent governance and operational interactions.

2.1 How to read the architecture

1. **Human users** access one HTTPS origin. Nginx sends page requests to Next.js and rewrites `/api/*` to FastAPI.
2. **Machine clients** use separately issued HTTP Basic credentials against `/api/v1`; scope and resource checks occur on every request.
3. **FastAPI** owns application contracts, parsing orchestration, role checks, release projection, resource filtering, and audit writes.
4. **PostgreSQL** stores searchable identifiers, normalized JSON sections, users/sessions, ownership, grants, protocols, and audit history.
5. **Persistent volumes** preserve original workbooks and SOP/protocol files separately from container images and database rows.
6. **CI/CD** deploys only after secret, static, dependency, tenant-isolation, SBOM, and container-image checks succeed.

2.2 Component Responsibilities

Layer	Confirmed technology	Responsibility and usage
Edge	Cloudflare-facing origin; Nginx stable	HTTPS termination at the origin, HTTP→HTTPS redirect, reverse proxy, request-size boundary, response security headers, and login/OTP/registration/reset throttling.
Frontend	Next.js 15.5.25, React 19, TypeScript, Tailwind CSS	Authenticated portal, public/team login modes, catalogue, backoffice, dynamic test routing, protocol browser, and API Explorer.
Scientific UI	Recharts 2.15, Lucide React	Test-specific charts, scientific tables, interactive run selection, images, status indicators, and client-side CSV/JSON downloads.
HTTP client	Axios	Same-origin <code>/api</code> requests with secure cookie credentials and generated <code>X-Request-ID</code> .
Backend	Python, FastAPI 0.136, Pydantic 2	REST routing, validation, user/session workflows, parser registry, publication projection, administrative APIs, and machine APIs.
Persistence access	SQLAlchemy 2 async + asyncpg	Async sessions, ORM models, transactions, query filtering, and PostgreSQL tenant context.
Scientific parsing	openpyxl	Reads workbook cells/sheets, normalizes values and dates, structures raw/processed/final/statistical sections, and records parser warnings.
Relational store	PostgreSQL 14	Metadata, JSON scientific records, users, sessions, organisations, protocol hierarchy, access assignments, credentials, release flags, and audit records.
File persistence	Host bind mounts / Docker volumes	Original research workbooks under <code>/app/data</code> , protocol documents under <code>/app/protocol_files</code> , encrypted access archives under <code>/var/backups/...</code> .
Runtime packaging	Docker Compose	Runs <code>db</code> , <code>backend</code> , <code>frontend</code> , <code>nginx</code> , and <code>public-access-retention</code> on an isolated bridge network.
Delivery	GitHub Actions + SSH deploy to IONOS	Security workflow gates deployment; server performs migrations, image builds, service restart, mount checks, docs-exposure checks, and health checks.

2.3 Broader context

Laboratory interoperability

Partners can continue producing familiar assay-specific workbooks. Dedicated parsers absorb layout differences and emit one application contract.

Scientific interpretation

Specialized viewers retain domain meaning that a generic JSON browser would lose: units, replicate context, plotted series, uncertainty, and statistical tables.

Governed dissemination

A record can be discoverable as public while raw, processed, final, statistical, or condition sections remain independently withheld.

Human and machine reuse

The portal optimizes comprehension; the `/v1` interface supports authorized pipelines without sharing browser sessions or administrator credentials.

Architectural boundary. PostgreSQL does not contain the uploaded workbook bytes. The `tests.file_path` value links a database record to the persistent research-data volume. Public responses always omit this internal server path.

3. Database and Storage Model

3.1 Hybrid persistence strategy

Relational identifiers

Primary keys, foreign keys, work package, CMS identifier, test name, tenant, publication state, timestamps, and grants support filtering and governance.

JSON scientific sections

Test-specific nested structures remain flexible without forcing every instrument and assay into one very wide relational schema.

Immutable source context

Excel and protocol files remain on protected persistent storage for provenance and possible reprocessing.

3.2 Central tests table

Column group	Columns	Meaning
Identity	<code>id</code> , <code>organisation_id</code>	Record key and owning tenant.
Scientific locator	<code>work_package_name</code> , <code>element_cms_id</code> , <code>test_name</code>	Human navigation and exact lookup triple used by <code>/tests/listings</code> .
Scientific payload	<code>test_details</code> , <code>raw_data</code> , <code>processed_data</code> , <code>final_results</code> , <code>statistical_analysis</code>	Five canonical JSON sections populated by a parser or JSON API.
Publication	<code>is_public</code> and five <code>release_*</code> booleans	Two-level disclosure: row discoverability plus independent section grants.
Outcome	<code>test_result</code>	Optional pass/fail summary independent of section JSON.
Provenance	<code>file_path</code> , <code>created_at</code> , <code>updated_at</code>	Source-workbook reference and lifecycle timestamps.

3.3 Principal relationships

```
# Scientific and protocol relationships
organisations 1 — * tests
organisations 1 — * categories 1 — * protocols 1 — * protocol_tests
protocol_tests --(work_package_name, element_cms_id, test_name)--> tests

# Interactive identity and public access history
users 1 — * sessions
users 1 — * public_data_access_events * — 1 tests

# User/API resource authorization
users 1 — 1 user_access_profiles
users * — * tests          via user_test_access
users * — * protocols      via user_protocol_access
organisations * — * tests/protocols via organisation_*_access
users/organisations 1 — * api_clients

# Governance workflow
organisations 1 — * developer_applications 1 — * access_requests
access_requests 1 — * approval_decisions
access_requests/applications 1 — * active_grants
organisations 1 — * audit_events
```

Protocol link detail. `protocol_tests` does not currently carry a foreign key to `tests.id`. It links to the scientific record through the unique locator triple. Its optional `display_name` changes presentation without changing the stable lookup values.

3.4 Active Application Table Catalogue

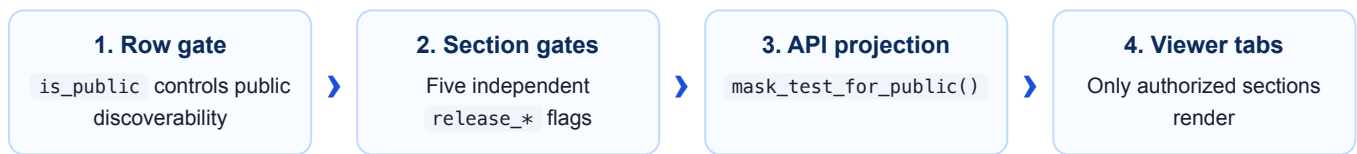
Domain	Table	Role	Key links / controls
Science	tests	Scientific metadata, five JSON sections, publication flags, source path.	→ organisations; public/RLS policies; section projection.
Identity	users	Local interactive accounts: name, email, bcrypt password, role, OTP seed, active state.	Unique email; roles admin/user/public_viewer.
Identity	sessions	Opaque browser sessions with expiry, active state, user agent, IP.	→ users; session ID stored in HttpOnly Secure cookie.
Public audit	public_data_access_events	Who viewed which test, through which route, with which released sections.	→ users/tests; 20-day live and encrypted-archive retention.
Machine access	api_clients	Current issued HTTP Basic client credentials.	→ organisations/users; bcrypt secret hash, scopes, disable/rotation version.
User grants	user_access_profiles	Global resource switches and platform tester designation.	PK/FK user; all_tests/all_protocols/all_files; audit tenant.
User grants	user_test_access	Explicit per-user test access.	Composite PK → users + tests.
User grants	user_protocol_access	Explicit per-user protocol access.	Composite PK → users + protocols.
Tenant	organisations	Tenant root and organization status.	Referenced by research, protocol, credential, grant, audit tables.
Tenant	organisation_memberships	Organization membership/role metadata.	→ organisations; historical keycloak_subject name.
Cross-access	organisation_test_access	Read a test without transferring ownership.	Composite PK → organisations + tests; RLS.
Cross-access	organisation_protocol_access	Read a protocol without transferring ownership.	Composite PK → organisations + protocols; RLS.
Catalogue	categories	Top-level protocol/navigation taxonomy.	→ organisations; 1:* protocols; ordered.
Catalogue	protocols	Protocol metadata and attached SOP file metadata.	→ categories/organisations; 1:* protocol_tests.
Catalogue	protocol_tests	Connects protocols to test locator triples.	→ protocols/organisations; unique test locator; display label.
API governance	api_definitions	Describes API products, version, classification, and scopes.	Unique name; active state.
API governance	developer_applications	Application registration and credential generation metadata.	→ organisations; legacy keycloak_client_id column name.
API governance	access_requests	Requested scopes, justification, status, and expiry.	→ organisation + application.
API governance	approval_decisions	Independent approval stages and reasons.	→ access_requests; unique role/stage per request.
API governance	active_grants	Issued scopes, expiry, and revocation metadata.	→ request/application/organisation.
Security audit	audit_events	Append-oriented, tamper-evident API/security activity.	→ organisations; sequence + previous/event hash.

3.5 Compatibility and Migration-Era Tables

The migration history also defines four tables from the earlier identity/access architecture. They are part of the schema history and may exist in deployed databases, but they are not the active authentication path described in Section 4.

Table	Original purpose	Current interpretation
<code>keycloak_identities</code>	Map an external identity-provider subject to an organization and legacy user.	Compatibility/cutover data. Active browser identity is local <code>users</code> + <code>sessions</code> .
<code>oauth_clients</code>	Registry for organization-bound OAuth machine clients.	Superseded operationally by local <code>api_clients</code> HTTP Basic credentials.
<code>api_scopes</code>	Scope definitions, classification, and approval requirement.	Governance reference; the active API issue UI uses an explicit allowlist matching route scopes.
<code>revocations</code>	Immutable revocation records for grants, OAuth clients, or memberships.	Compatibility/governance history; current local API clients also have immediate <code>is_active</code> disablement.

3.6 Publication and field projection



- A private test cannot retain active release grants; attempts fail closed or revoke the flags.
- Public viewers receive public rows only, with unreleased scientific sections removed.
- The fixed seven-field report header remains available for identification.
- Team users and administrators receive the complete authorized record.
- `file_path` is never returned in the public projection.

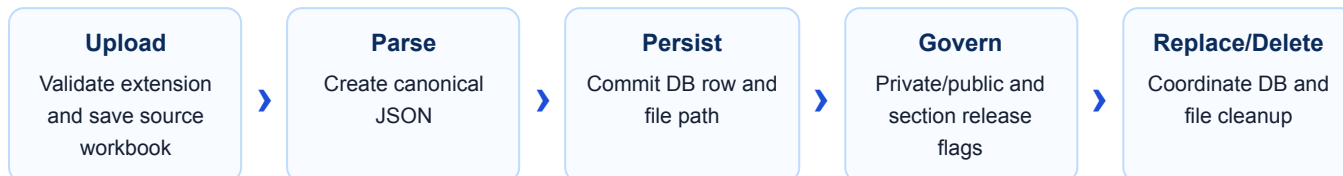
3.7 Tenant and resource controls

Defense in depth. Machine API reads combine (1) credential authentication, (2) scope checks, (3) explicit user/resource profile checks, (4) transaction-local organization context, (5) PostgreSQL RLS policies, and (6) audit events. Transaction-local `set_config(..., true)` prevents tenant context leaking through the connection pool.

Operational migration note. Repository guidance states that legacy ownership reconciliation must be completed before the production runtime database connection is switched from the current owner role to the least-privilege, RLS-enforcing `chemat_app` role. RLS policies and CI tests exist, but operators must verify the live connection role and ownership migration status rather than infer it from source code.

3.8 Data Lifecycle, Audit, and Recovery

Scientific record lifecycle



Access and audit data

Record	Captured information	Retention/control
Browser session	Opaque session ID, user, creation/expiry, active flag, user agent, IP.	Default seven-day expiry; logout invalidates; expired sessions are cleaned during session creation.
Public data access event	User identity snapshot, test locator, owning organization, access level, released sections, request ID, route, timestamp.	20 days. Hourly worker encrypts and verifies archives before deleting expired live rows; expired archive entries are also pruned.
Machine/API audit event	Actor subject/client, action, resource, outcome, timestamp, safe metadata, hash chain.	Append-oriented database permissions and HMAC/hash-chain tamper evidence.
Source research file	Original uploaded workbook.	Persistent host bind mount; replacement deletes old file only after DB update succeeds; failed parse/create cleans the new file.
Protocol file	SOP document and DB metadata (name, MIME, size, path).	Protected persistent mount; authorized download with safe path resolution and no-store headers.

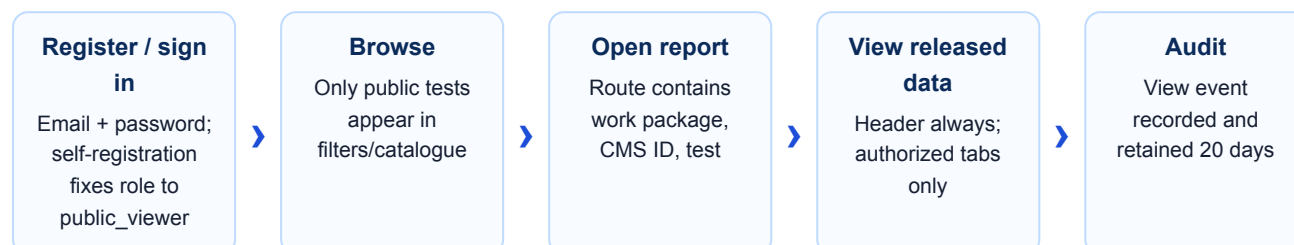
Operational recovery controls

- PostgreSQL data persists in the `pgdata` Docker volume.
- Research workbooks and protocol files use host bind mounts so image rebuilds do not erase them.
- The deploy script refuses to proceed over tracked server edits, applies reviewed migrations, verifies mounts, rebuilds images, and checks direct plus Nginx health.
- API documentation endpoints are fail-closed and the deployment explicitly verifies that `/docs`, `/redoc`, and `/openapi.json` are not published.

4. Process Flow from the User Point of View

4.1 Public viewer

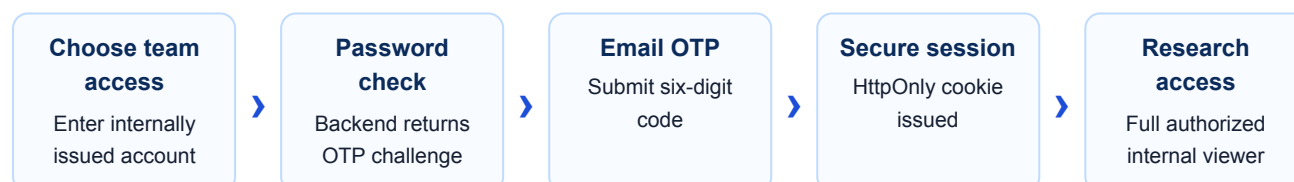
Public access journey



The browser session is created immediately after successful public registration or password authentication. Public viewers do not complete the privileged OTP step. The backend—not the browser—filters public records and projects sections, so hiding tabs is a usability layer rather than the security boundary.

4.2 Team user

Consortium/team journey



Team users can browse internal resources available to their role/access assignments. Specialized test viewers provide full scientific presentation rather than public-section masking.

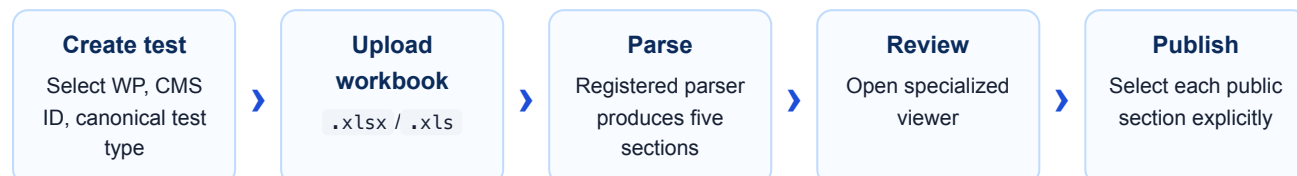
4.3 Password and session behavior

- Passwords are bcrypt-hashed; schemas require at least 12 characters for account creation/reset.
- Privileged `user` and `admin` roles require emailed OTP after password verification.
- The session cookie is HttpOnly, Secure by default, SameSite=Lax, and has a seven-day maximum age.
- Unsafe cookie-authenticated requests require an allowed `Origin`; responses receive request IDs, no-store, and security headers.

4.4 Administrator and API Client Flows

Administrator: ingest and publish

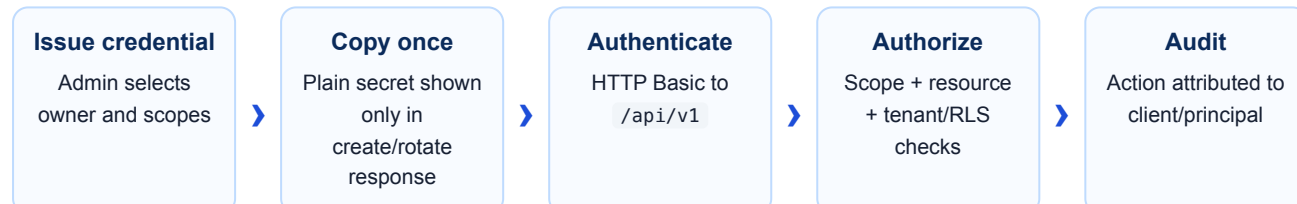
Backoffice test-management journey



Administrators can create, update, delete, filter, publish, and unpublish tests; manage users and organisations; assign test/protocol/file access; inspect public access history; and issue or disable API credentials. Making a test private revokes all five section grants.

Authorized API client

Machine-to-machine journey



Available issue-time scopes are:

tests:read experimental-data:read protocols:read protocol-files:download files:navigate

The client secret is stored only as a bcrypt hash. Rotation invalidates the previous secret immediately; disabling an API client affects its next request. Disabling a human user does not implicitly disable their API clients, so offboarding includes the dedicated “disable all” operation.

Separation of channels. Browser users authenticate with a session cookie. Machine clients authenticate with HTTP Basic credentials. API Explorer is a convenience interface for issued API credentials; it does not reuse or elevate the browser’s team/admin session.

5. TB Test: End-to-End Coding Logic

The Trypan Blue (TB) implementation is representative because it contains structured conditions, repeated raw runs, processed values, final replicate statistics, an optional statistical sheet, charts, images, downloads, and direct support for partial public releases.

5.1 File and component map

Stage	Source	Key symbol
Parser	<code>backend/parsers/tb.py</code>	<code>TBParser</code> , <code>parse_excel_tb()</code>
Registry/upload	<code>backend/api/controllers/test.py</code>	<code>PARSERS["TB"]</code> , <code>save_and_parse()</code> , <code>build_payload_from_parse()</code>
Persistence	<code>backend/api/models/test.py</code>	<code>Test</code> JSON columns and release flags
Read service	<code>backend/api/services/test.py</code>	<code>get_listings()</code> , <code>mask_test_for_public()</code>
Dynamic dispatch	<code>frontend/.../RoleAwareTestViewer.tsx</code>	<code>SPECIALISED_VIEWERS["tb"]</code>
Viewer	<code>frontend/src/components/tests/tb/page.tsx</code>	<code>TBDataViewer</code>

5.2 Workbook contract

Sheet/pattern	Parser behavior	Output
<code>Test_conditions</code>	Reads key/value labels and fixed cells for work package, scientists, material, dispersion, cell line, treatment, and replicate metadata.	<code>test_details</code> plus <code>replication_metadata</code>
<code>Raw_data_*</code>	Discovers all matching sheets; extracts test/plate labels, assay metric, chamber measurements, protocol details, instrument serial, and notes.	List of raw run blocks
<code>Processed_data_*</code>	Matches suffixes to raw sheets; reads condition labels and derived cell-death values.	List of processed run blocks
<code>Final_results</code>	Reads sample/endpoint, four conditions, three replicate rows, mean row, and standard-deviation row.	Final result object
<code>Statistics</code> (optional)	Reads software/test, ANOVA and Brown–Forsythe summaries, ANOVA table, data summary, and Tukey post-hoc comparisons.	Statistical analysis object; otherwise <code>available:false</code>

5.3 Backend Parser Logic

1 Open and normalize

`TBParser.__init__()` loads the workbook with formulas resolved (`data_only=True`) and selects `Test_conditions` . Helpers normalize human labels into snake-case keys, parse Excel/date variants, replace decimal commas, and return `None` for unusable numeric values.

```
normalize_key(label): lowercase → replace non-alphanumeric runs with "_"
excel_date_to_string(value): datetime | Excel serial | known string formats → YYYY-MM-DD
_safe_float(value): decimal comma → decimal point → float; invalid → None
_safe_str(value): trim non-empty values
```

2 Extract typed scientific structures

Dataclasses define the parser's output vocabulary— `WorkPackageData` , `MaterialData` , `DispersionData` , `CellLineData` , `TreatmentData` , raw/processed blocks, and final results. Extraction methods return dictionaries using `asdict()` .

```
extract_work_package_data()
    scan first six columns for label/value rows
    map normalized labels to wp, partner, full name, acronym, type, endpoint, SOP
    read scientist names/emails; retain email only when regex-valid

extract_material_data()
    map Test_conditions rows 26–43 to identifiers, chemistry, batch and stock metadata

extract_raw_data()
    for each sheet whose name starts with "raw_data":
        find matching "Processed data_<suffix>"
        collect chamber labels and % cell death from columns D:G
        collect protocol/instrument values and assay notes
```

3 Assemble one canonical payload

```
parse_all_data() → {
    "test_details": {work_package, material, dispersion, cell_line, treatment},
    "replication_metadata": [...],
    "raw_data": raw_runs,
    "replications": raw_runs,          # compatibility key consumed on upload
    "processed_data": processed_runs,
    "final_results": final_summary,
    "statistical_analysis": statistics,
    "parser_warnings": [...]          # only when present
}
```

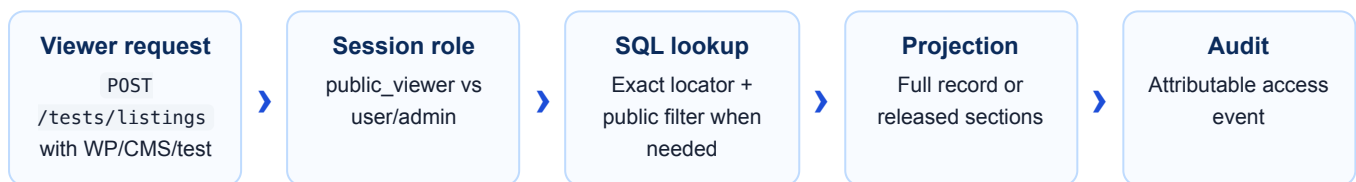
Important mapping. The upload controller's `build_payload_from_parse()` writes `file_data["replications"]` into the database `raw_data` column. TB deliberately emits both `raw_data` and `replications` with the same list for compatibility with this canonical upload path.

5.4 Upload, Persistence, and Read Projection

Administrator upload sequence

1. `POST /tests/` requires the administrator role.
2. `validate_excel_filename()` accepts only `.xlsx` or `.xls`.
3. `get_parser("TB")` resolves `parse_excel_tb` from the centralized 19-test registry.
4. `save_uploaded_file()` sanitizes the timestamped name and writes the workbook under the persistent research-data mount.
5. The parser runs. Any parser failure deletes the just-saved file and returns a non-sensitive 400 response.
6. `clean_for_json()` recursively converts dates, decimals, tuples, and unsupported values into JSON-safe primitives.
7. `TestService.create_test()` rejects a duplicate locator triple, validates private/release consistency, commits the row, and refreshes it.
8. If database creation fails, the controller removes the saved workbook so no orphan is left.

Read sequence



Public projection pseudocode

```

mask_test_for_public(test):
  return {
    identity: id + work_package_name + element_cms_id + test_name,
    test_details: full section IF release_test_details
                  ELSE seven-field public header allowlist,
    raw_data: test.raw_data IF release_raw_data ELSE null,
    processed_data: test.processed_data IF release_processed_data ELSE null,
    final_results: test.final_results IF release_final_results ELSE null,
    statistical_analysis: value IF release_statistical_analysis ELSE null,
    release_flags: boolean values,
    file_path: null
  }

```

Why projection is server-side. A public user must never receive unreleased values and merely have them hidden by React. The response itself is minimized before serialization. Viewer tab filtering then mirrors the authoritative release flags.

5.5 TB Frontend Viewer Logic

Fetch and state

`TBDataViewer` receives the decoded route values and requests the exact test record. React state tracks the response, loading/error state, active tab, and selected run.

```
useEffect([work_package, element, test]):
  POST /tests/listings { work_package_name, element_cms_id, test_name }
  setData(response.data)

state = { data, loading, error, activeTab, selectedRun }
```

Release-aware tab construction

TB is directly safe for partial public releases. It derives available tabs from response values; Test Conditions additionally checks `release_test_details !== false` because the public response may contain only the header metadata envelope.

```
Test Conditions    → release_test_details is not false AND test_details exists
Raw Data          → raw_data exists
Processed Data     → processed_data exists
Statistical Analysis → statistical_analysis exists
Final Results      → final_results exists

IF active tab is unavailable: select the first available tab
```

Derived visualization data

Memoized model	Transformation	UI result
<code>safeRawData</code>	Accept arrays only; otherwise empty list.	Safe run selector, raw table, and chart.
<code>rawBarChartData</code>	Each chamber → {chamber, value}.	Bar chart of percent cell death.
<code>safeProcessedData</code>	Accept arrays only and select current/first run.	Condition/value processed chart and table.
<code>finalMeanChartData</code>	Conditions joined with mean and SD maps.	Final mean ± SD visualization.
<code>finalReplicateSeries</code>	Replicate rows assigned stable keys/colors.	Per-replicate series alongside summary.
Statistics sections	Read optional summary/table arrays defensively.	ANOVA, Brown–Forsythe, data summary, and Tukey tables.

Presentation and export

- The always-visible header shows work package, CMS ID, ERM ID, full name, acronym, type, endpoint, outcome, and SOP.
- Test Conditions renders material, dispersion, cell line, treatment, and metadata tables only when released/authorized.
- Each data section has explicit empty states rather than assuming values exist.

- `downloadTable()` reads rendered table cells, quotes CSV values safely, and downloads a section-specific file.
- Configured TB images use normalized CMS identifiers and provide display, missing-image fallback, and download links.

5.6 End-to-End TB Trace

#	Input/state	Code decision	Output/next stage
1	Admin selects TB and an Excel file.	Canonical value <code>TB</code> matches parser and viewer registries.	Multipart request to <code>POST /tests/</code> .
2	Filename has supported extension.	Sanitize and save to persistent research volume.	Stable source file path.
3	Workbook sheets and cells.	<code>TBParser</code> normalizes values and extracts each domain.	Canonical nested dictionary.
4	Parser dictionary.	Controller maps compatibility <code>replications</code> to DB <code>raw_data</code> ; cleans JSON values.	Validated <code>TestCreate</code> .
5	Locator + payload + release state.	Service checks duplicates/private grants and commits transaction.	<code>tests</code> row plus file reference.
6	User browses <code>WP/material/test</code> .	Catalogue/listings filter by interactive role.	Dynamic route <code>/wp/{element}/TB</code> .
7	Viewer requests exact locator.	Public response is projected; private authorized response is complete.	Safe TB response and access event.
8	TB React component receives response.	Build tabs and memoized chart/table models from available sections.	Scientific report, charts, tables, images, downloads.

Failure behavior

Failure	Behavior
Unknown test type	Upload receives 400; dynamic viewer reports no configured viewer.
Unsupported file extension	Upload rejected before parsing.
Workbook/sheet cannot be loaded	Parser logs the internal error; saved file is removed; client receives safe processing error.
Optional Final results missing	Parser warning plus structurally valid empty final result.
Statistics missing	<code>{available:false, notes:...}</code> ; viewer renders an unavailable/empty state.
Duplicate WP/CMS/test triple	Service returns conflict rather than creating an ambiguous record.
Public request for private test	404 to avoid exposing record existence.
Unreleased section	Section payload removed server-side and corresponding tab hidden.

6. Operations and Extension Guide

6.1 Adding a new test type

1. Define the canonical test identifier and keep it identical across create options, parser registry, URL dispatch, and viewer registry.
2. Create a parser under `backend/parsers/` that returns all five canonical sections (use explicit empty/unavailable structures where a section is unsupported).
3. Add the parser function to `PARSERS` in `backend/api/controllers/test.py`.
4. Create a null-safe TypeScript viewer under `frontend/src/components/tests/`.
5. Register the viewer in `RoleAwareTestViewer.tsx` and add catalogue display metadata.
6. Add the test option to `CreateTestModal.tsx`.
7. Test every independent public section flag and all relevant combinations; never infer a section's release from another section's presence.
8. Run parser tests, public projection tests, backend suite, ESLint, TypeScript, Next.js build, Compose validation, and security workflow.

6.2 Parser design rules

- ✓ Normalize spreadsheet variability at the parser boundary.
- ✓ Use safe conversions and preserve missing values as `null / None`.
- ✓ Return stable typed keys; do not make viewers understand workbook coordinates.
- ✓ Record optional/missing scientific sections explicitly rather than crashing.
- ✓ Avoid logging complete scientific payloads in production; treat source data according to classification.

6.3 Database change rules

- ✓ Use reviewed, idempotent migrations outside development; production auto-DDL must remain off.
- ✓ Back up and restore-test before ownership/backfill or destructive changes.
- ✓ Preserve fail-closed behavior when tenant context, release state, or credentials are missing.
- ✓ Keep heavy JSON out of list/index queries; fetch full payload only for one authorized record.
- ✓ Coordinate DB rows and persistent files to avoid dangling paths or orphan files.
- ✓ Do not store plaintext passwords, client secrets, OTPs, authorization headers, or tokens in logs/audit metadata.

6.4 Verification commands

```
# Backend
docker run --rm -v "$PWD/backend:/app:ro" -w /app \
  chemat-sustain-backend:test-deps python -m pytest -q

# Frontend
cd frontend && npm run lint && npx tsc --noEmit && npm run build

# Deployment configuration and diff integrity
docker compose config --quiet
git diff --check
```

7. Glossary and Source Map

Glossary

Term	Meaning in this system
CMS identifier	CheMatSustain material identifier used in record lookup and routes, e.g. <code>CMS_30a_CH_TER</code> .
ERM identifier	External/reference material identifier stored within material test details.
Canonical sections	The five normalized scientific JSON groups shared across test types.
Public test	A row with <code>is_public=true</code> ; section flags still determine disclosed scientific content.
Public viewer	Authenticated least-privilege interactive role that can read only public projected records.
Team user	Interactive <code>user</code> role with password + OTP and authorized internal record access.
API client	Issued machine credential with hashed secret, scopes, and resource/tenant assignments.
RLS	PostgreSQL Row-Level Security, using transaction-local organization/access context.
Header projection	Seven allowlisted identification fields retained when detailed Test Conditions are unreleased.
Protocol test	A navigation-tree link from a protocol to a test locator triple, not the scientific data row itself.

Authoritative source map

Concern	Primary source
Application assembly and middleware	<code>backend/app.py</code>
Database engine/session	<code>backend/utls/db.py</code>
ORM model definitions	<code>backend/api/models*.py</code> , <code>backend/api/models/*.py</code>
Reviewed schema evolution	<code>backend/migrations/</code> and <code>backend/migrations/sql/</code>
Scientific upload and parser registry	<code>backend/api/controllers/test.py</code>
Test query/public projection	<code>backend/api/services/test.py</code>
TB parser	<code>backend/parsers/tb.py</code>
TB viewer	<code>frontend/src/components/tests/tb/page.tsx</code>
Viewer dispatch/public fallback	<code>RoleAwareTestViewer.tsx</code> , <code>PublicReleasedDataViewer.tsx</code>
Interactive identity	<code>backend/api/controllers/user.py</code> , <code>backend/utls/auth.py</code>
Machine identity and scopes	<code>backend/security/auth.py</code> , <code>router_api_clients.py</code> , <code>router_phase1.py</code>
Tenant context	<code>backend/security/tenant.py</code> and RLS migrations
Retention	<code>backend/scripts/public_access_retention.py</code>
Production topology	<code>docker-compose.yml</code> , <code>nginx/default.conf</code>
Security/deployment gate	<code>.github/workflows/security.yml</code> , <code>deploy.yml</code> , <code>scripts/deploy.sh</code>

System summary. CheMatSustain uses a hybrid database architecture to preserve scientific flexibility without sacrificing governance: source workbooks remain durable, parser-specific knowledge is isolated in backend modules, canonical JSON enables common APIs, specialized viewers retain scientific meaning, and layered identity/release/tenant controls constrain who can see what.

